

Table of contents

1	Performance Benchmarking	2
1.1	Real-time Metrics	2
1.2	Understanding the Bottlenecks	2
1.2.1	The Stages of Execution	3
1.3	Interactive Performance Sandbox	3
1.4	Potential Optimizations	4

Chapter 1

Performance Benchmarking

This page allows you to evaluate how the Hybrid Fluid-Light Transport engine performs on your specific hardware.

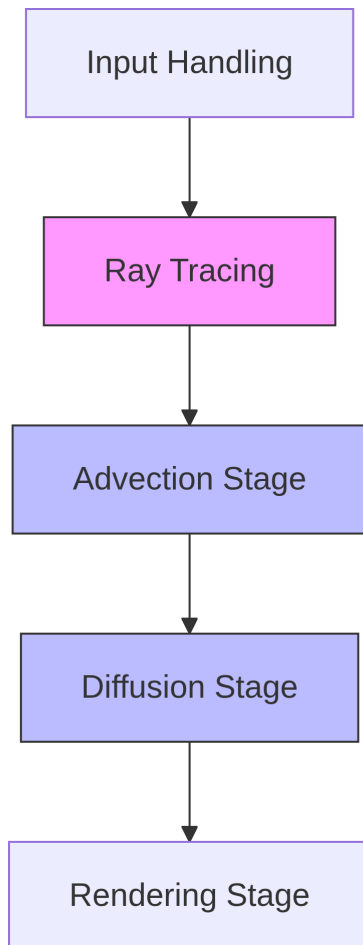
1.1 Real-time Metrics

The simulation below runs a standardized “Stress Test” to measure the limits of the Cellular Automata (CA) and Ray Tracing stages.

1.2 Understanding the Bottlenecks

Treatment of light as a fluid is computationally demanding. Unlike standard GPU-based shaders, our engine currently runs entirely on the **CPU** within the JavaScript main thread.

1.2.1 The Stages of Execution



1. **Ray Tracing:** Calculates where light enters the system. Scaling is $O(\text{Ray Count} \times \text{Resolution})$.
2. **Fluid Stages:** Moves energy across the grid. Scaling is $O(\text{Grid Width} \times \text{Grid Height})$.

1.3 Interactive Performance Sandbox

Adjust the parameters below to see their theoretical impact on your frame budget (16.6ms for 60 FPS).

```
//| echo: false
viewof res = Inputs.select([
  {label: "Low (160x90)", value: 14400},
```

```

    {label: "Medium (320x180)", value: 57600},
    {label: "High (640x360)", value: 230400}
  ], {label: "Grid Resolution"})

viewof rays = Inputs.range([8, 128], {value: 64, step: 8, label: "Rays per Frame"})

msPerPixel = 0.00007 // Estimated CPU time per pixel advection/diffusion
msPerRay = 0.05      // Estimated CPU time per ray cast

totalTime = (res * msPerPixel) + (rays * msPerRay)
fps = Math.min(60, 1000 / totalTime)

md`
### Predicted Frame Time: **${totalTime.toFixed(2)}ms**
### Predicted FPS: **${fps.toFixed(1)}**

${totalTime > 16.6 ?
  " **Performance Warning**: This configuration exceeds the 16.6ms budget. Expect stutter
  " **Optimal**: This should run smoothly at 60 FPS."}
`

```

1.4 Potential Optimizations

To improve performance in future versions, we are exploring:

- **Web Workers:** Moving the Diffusion and Advection loops to background threads.
- **SIMD (Single Instruction, Multiple Data):** Using Float32Array with vectorized operations via WebAssembly.
- **GPU Acceleration:** Ports of the CA logic to WebGL/WebGPU compute shaders.